MIDTERM: DSEB65A

Question 1.(5 points). Create a Jupyter Notebook file with the name **id_fullname.ipynb**. In the first block of the file, fill in the following fields completely in markdown mode.

- **Class:**
- **Fullname:**
- **ID:**

Your first block code should be: **id = <your_student_id>**

Question 2.(40 points)

a) (20 points) Write a class called **TextAnalyzer**. This class should have the following methods:

- **init__(self, file_name)**: Initializes the object by reading the content from the given file (in folder Text_file) (encoding="utf8").
- **normalize_text(self, lower=True)**: Uses a regular expression to remove all special characters, leaving only alphanumeric characters and spaces. The optional parameter lower (default: True) will convert all words to lowercase if set to True. Returns a list of normalized words.
- **count_word_frequency(self, words, decrease=True)**: Takes a list of words and an optional parameter decrease (default: True) to sort word frequencies in descending order. Returns a dictionary of word frequencies.
- Create an object from class **TextAnalyze**r with file_name = get_file(id) (the function get_file in midterm_dseb65a.py). Use the normalize_text and count_word_frequency methods to display the word frequency dictionary in descending order.

b) (20 points) Create a new class called ExtendedTextAnalyzer that inherits from TextAnalyzer and adds the following methods:

- **get_dictionary(self)**: Returns a sorted list of unique, normalized words from the text (as processed by normalize_text), ordered alphabetically.
- **convert2vector(self)**: Converts each sentence in the file's content to a vector. Each sentence corresponds to a row in the resulting matrix, where the vector represents the frequency of each word in the get_dictionary list for that sentence.
- **find_most_similar_sentences(self)**: Calculates the cosine similarity between all pairs of sentence vectors (from convert2vector) and returns the pair of sentences with the highest cosine similarity score.
- Give all the results of the methods of the ExtendedTextAnalyzer class with the file_name in part a.

Question 3. (30 points). A = get_matrixt(id) (import the function get_matrix from midterm_dseb65a.py)**.** Create a function to

1. Count the entries larger than the mean of all entries of matrix A
2. Find the most frequency of the entries in matrix A
3. Count the number of the prime numbers of A
4. Sum of all entries of matrix A, which are product by two primes

Question 4.(30 points) test = get_test(id) (import the function get_test from midterm_dseb65a.py)

a) (10 points) Some values in test are "dseb65a" or in the form of letter digits; write a function **fix** to replace "dseb65a" by 0.67 and convert the letter digit to a float number (ex: "0.33" to 0.33).

b) (10 points) Write a function called grade to sum the scores from "Q1" to "Q15" for each student. The function returns a dictionary with key is the id and values is the grade.

c) (5 points) Write a function called get_topk(…,k=10) to get the top 10 people with the highest scores (using id_dict in midterm_dseb65a.py).

d) (5 points) Write a function get_highets_in_kfastest(…,k=10) to find the person with the highest score among the 10 fastest people.

----END----

Pass:dseb65a_midterm0511